Práctica 2: Aprendizaje No Supervisado

Izan Cuesta Corbí
DNI: 48804930B
Universidad de Alicante – Escuela Politécnica Superior
Aprendizaje Avanzado – Curso 2025/2026
Email: icc87@alu.ua.es

Resumen

En esta práctica se abordan técnicas de Aprendizaje No Supervisado utilizando tres tipos de conjuntos de datos: escenarios sintéticos, el dataset Digits proyectado con UMAP y un dataset de anomalías proveniente de KDD Cup 99. Para ello, he evaluado los algoritmos K-Means, Spectral Clustering, DBSCAN y OPTICS en tareas de clustering, así como Gaussian Mixture Models (GMM) e Isolation Forest para la detección de anomalías. La estrategia experimental consistió en el ajuste exhaustivo de hiperparámetros de cada modelo y su evaluación mediante métricas internas (como Silhouette) y externas (como ARI, NMI, Precision, Recall y F1), apoyándose fuertemente en análisis visuales mediante proyecciones, gráficas de k-distances y reachability plots. Los resultados determinan que, en los escenarios sintéticos, OPTICS es el algoritmo más versátil al adaptarse a cualquier geometría, superando las limitaciones de densidad de DBSCAN y las fronteras lineales de K-Means. En el dataset Digits, OPTICS se consagró también como el mejor método con un ARI de 0.9455 al manejar eficazmente el ruido. Finalmente, en la detección de anomalías, Isolation Forest resultó ser el único detector verdaderamente viable y competitivo, logrando maximizar la métrica crítica de Recall (0.9959) frente a la incapacidad demostrada por GMM y DBSCAN en este entorno desbalanceado.

Palabras clave: Aprendizaje no supervisado, Clustering, DBSCAN, OPTICS, Spectral Clustering, GMM, Isolation Forest, Detección de anomalías.

Índice

1. Parte 1: Clustering **3**
1.1. K-Means (Referencia) . . . 3
1.1.1. Escenarios sintéticos . . . 3
1.1.2. Digits (K=10) . . . 3
1.2. Clustering Espectral . . . 4
1.2.1. Escenarios sintéticos . . . 4
1.2.2. Digits (K=10) . . . 5
1.3. DBSCAN . . . 6
1.3.1. Escenarios sintéticos . . . 6
1.3.2. Digits . . . 7
1.4. OPTICS . . . 8
1.4.1. Escenarios sintéticos . . . 8
1.4.2. Digits . . . 9
1.5. Comparativa Final – Parte 1 . . . 10
1.5.1. Escenarios sintéticos . . . 10
1.5.2. Digits . . . 11

2. Parte 2: Detección de Anomalías **13**
2.1. Dataset y Exploración . . . 13
2.2. Gaussian Mixture Models (GMM) . . . 13
2.2.1. Selección de componentes y covarianza . . . 13

2.2.2. GMM como detector de anomalías 14
2.3. Isolation Forest 14
2.4. DBSCAN como Detector de Anomalías 16
2.5. Evaluación Final – Parte 2 16

3. Conclusiones **18**

1. Parte 1: Clustering

1.1. K-Means (Referencia)

1.1.1. Escenarios sintéticos

Se aplicó K-Means con el número real de clusters a cada escenario sintético.

Tabla 1: K-Means sobre escenarios sintéticos

Escenario	K	ARI	NMI	Silhouette
Blobs	3	1.0000	1.0000	0.8441
Moons	2	0.4785	0.3803	0.4961
Circles	2	-0.0006	0.0000	0.3544
Densidad variable	3	0.8150	0.7853	0.6488
Anisotrópico+ruido	3	0.8507	0.8188	0.6812

K-Means únicamente funciona de manera óptima en el escenario de Blobs, logrando unas métricas perfectas (ARI = 1.0 y NMI = 1.0). Por el contrario, el algoritmo presenta grandes fallos en los escenarios de Circles (NMI = 0.0 y ARI = -0.0006, resultado equivalente a una asignación aleatoria) y Moons (ARI = 0.4785 y NMI = 0.3803). Para el resto de escenarios, obtenemos un rendimiento subóptimo, siendo Densidad variable (ARI = 0.8150 y NMI = 0.7853) y Anisotrópico+ruido (ARI = 0.8507 Y NMI = 0.8188).

El fracaso de Circles y Moons se debe a que K-Means utiliza la distancia euclídea pura para asignar cada punto a su centroide más cercano, lo que provoca que el algoritmo divide el espacio trazando fronteras lineales. Al tratarse de dos escenarios con con formas geométricas no convexas, K-Means es incapaz de envolver una forma dentro de otra, por lo que simplemente corta el conjunto de datos por la mitad con una línea recta.

La otra métrica interna que hemos sacado en el análisis ha sido Silhouette, la cual tener un valor alto no podemos decir que equivalga a una buena segmentación. Por ejemplo, en el caso de Moons, tenemos que su ARI es de 0.4785, mientras que su Silhouette es de 0.4961 (mayor que ARI). Pese a que sea un resultado mayor, aún así seguimos sabiendo que el algoritmo se ha equivocado por completo al agrupar la forma real de las lunas. Esto se debe a que Silhouette también se basa en distancias euclídeas, y sigue siendo más importante tener controlados los resultados de las métricas externas como ARI.

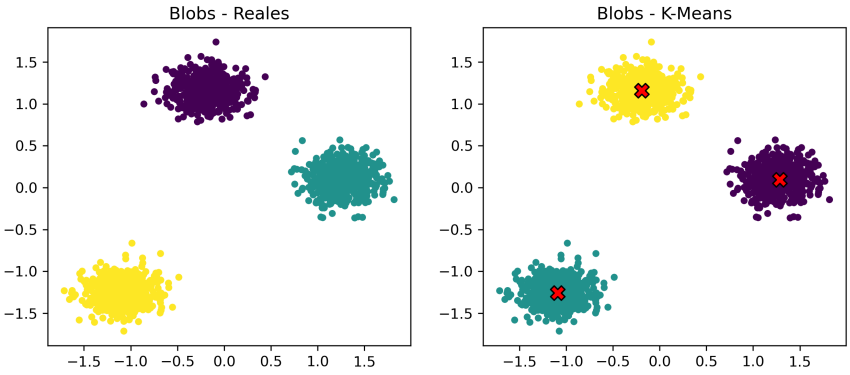


Figura 1: Ejemplo de K-Means sobre el escenario Blobs

1.1.2. Digits (K=10)

En el ejemplo de Digits, K-Means logra capturar la estructura principal de los 10 dígitos.

En la gráfica de puntuación silhouette, se confirma la teoría de manera visual. Sin embargo, el $K = 10$ en esta gráfica sería el resultado subóptimo, ya que el pico más absoluto se encuentra en

Tabla 2: K-Means sobre Digits con K=10

K	ARI	NMI	Silhouette	Inercia
10	0.8248	0.8912	0.7737	56.7528

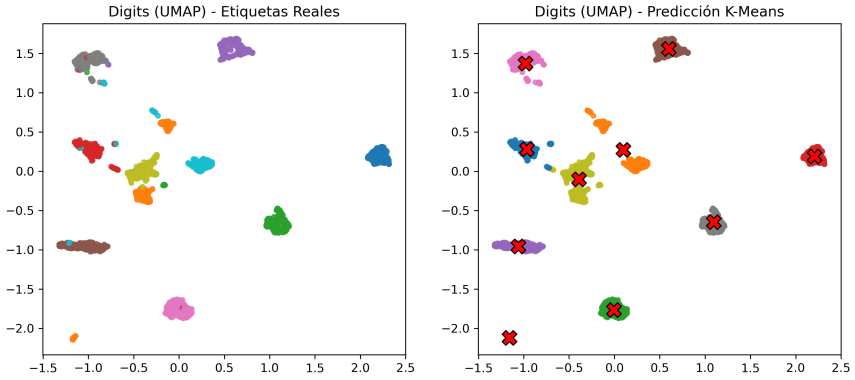


Figura 2: K-Means (K=10) sobre Digits – proyección UMAP

$K = 11$ (con un valor silhouette superior a 0.8). Si nos fijamos en las gráficas sobre Digits, podemos observar como hay un grupo de puntos naranja (el central) en la predicción kmeans que realmente por su posición deberían de ser dos grupos. Al haberle obligado al algoritmo a obtener 10 clusters exactos, estos dos grupos se han tenido que juntar para cumplir con el requisito.

En cuanto a la gráfica del método del codo, corroboramos la idea de la anterior gráfica. Podemos observar una caída en picado de la inercia (la suma de las distancias al cuadrado dentro de los clusters) a medida que aumentamos el número de grupos. Al llegar a $K = 11$, la curva cambia de pendiente (el codo), volviéndose casi plana y lineal (pese a que la linea se traze en 10 clusters). Esto significa que añadir un clúster 12° ya no aporta una mejora significativa y K-Means ya tendría los 11 conjuntos principales.

Si nos fijamos puramente en los resultados, el ARI (0.8248) nos indica que la segmentación no es perfecta (lo que hemos descubierto visualmente gracias a las gráficas).

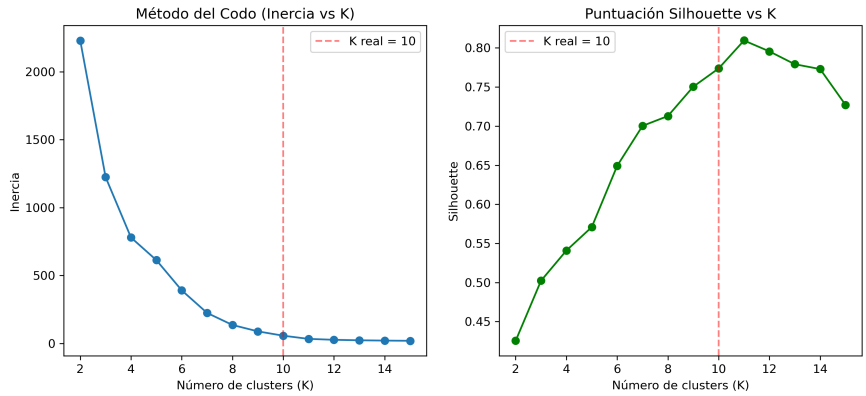


Figura 3: Método del Codo y Puntuación Silhouette

1.2. Clustering Espectral

1.2.1. Escenarios sintéticos

El Spectral Clustering supera enormemente a K-Means en los escenarios de geometrías no convexas: Moons y Circles. Mientras que K-Means fracasaba por intentar trazar líneas rectas (ARI en 0 y 0.47

Tabla 3: Spectral Clustering sobre escenarios sintéticos

Escenario	**n_neighbors**	**ARI**	**NMI**	**Silhouette**
Blobs	5	0.5695	0.7258	0.1624
Blobs	15	1.0000	1.0000	0.8441
Moons	5	0.0001	0.0090	-0.0687
Moons	15	1.0000	1.0000	0.3935
Circles	5	0.9814	0.9618	0.1092
Circles	15	1.0000	1.0000	0.1127
Densidad variable	5	0.8790	0.8567	0.6428
Densidad variable	15	0.9567	0.9293	0.6406
Anisotrópico+ruido	5	0.7815	0.7453	0.6495
Anisotrópico+ruido	15	0.8405	0.8076	0.6756

respectivamente), Spectral logra un ARI perfecto de 1.0000 (usando 15 vecinos). Además, muestra un rendimiento notablemente superior en el escenario de Densidad variable (obteniendo un ARI de 0.9567 frente al 0.8150 de K-Means). Esto demuestra que Spectral no necesita que las agrupaciones tengan varianzas similares o sean esféricas, sino que le basta con que exista continuidad espacial entre los puntos.

Para la exploración de los posibles resultados, hemos utilizado el parámetro n_neighbours. Este, afecta drásticamente al resultado porque determina cómo se construye el grafo de similitud subyacente. Al usarlo, el algoritmo dibuja una línea entre cada punto y sus N vecinos más cercanos.

En la práctica, podemos ver como en todos los escenarios, el uso de n_neighbours = 15 es muy superior a los casos en los que se usan 5. Si observamos la tabla, con n_neighbors=5, el escenario Moons colapsa por completo (su ARI pasa de 1.0 a 0.0001), y aunque Circles aguanta mejor (0.98), Blobs también cae (0.56). Esto ocurre porque al mirar solo a los 5 vecinos más cercanos, el grafo se fragmenta. En lugar de ver una luna entera conectada, el algoritmo ve grupos aislados de 5 puntos. Al intentar encontrar los 2 clusters principales ($K = 2$), el algoritmo se pierde al no haber suficientes puentes de conexión a lo largo de la forma geométrica.

1.2.2. Digits (K=10)

Tabla 4: Spectral Clustering sobre Digits (K=10)

affinity	**n_neighbors / γ**	**ARI**	**NMI**	**Tiempo (s)**
nn	5	0.2416	0.5422	0.5823
nn	15	0.1120	0.4724	0.2056
nn	30	0.3521	0.6883	0.1654
nn	50	0.6433	0.7993	0.1578
rbf	0.1	0.7861	0.8865	0.3823
rbf	1.0	0.8104	0.8798	0.3460
Mejor	1.0	**0.8104**	**0.8798**	**0.3460**

Para el análisis de resultados del clustering espectral sobre Digits, los resultados obtenidos nos dicen claramente que la afinidad RBF funciona significativamente mejor que NN. Mientras que la configuración óptima de vecinos más cercanos (nn=50) apenas logra un ARI de 0.6433, ambas configuraciones de RBF superan esa marca, alcanzando el pico máximo absoluto en gamma=1.0 con un ARI de 0.8104 y un NMI de 0.8798. Esto nos indica que, para la forma concreta en la que UMAP ha proyectado los dígitos en el espacio 2D, una RBF captura mucho mejor los límites de las islas que una conexión discreta por vecinos.

Pese a que construir una matriz RBF es muy costoso computacionalmente $O(N^2)$ porque calcula la distancia de cada punto contra todos los demás puntos, mientras que nearest_neighbors puede optimizarse para buscar solo conexiones locales, si nos fijamos en la columna del tiempo, vemos que en la práctica los tiempos son muy similares. La configuración ganadora de RBF tarda 0.42 segundos,

mientras que la mejor de nn tarda 0.31 segundos (e incluso configuraciones peores de nn tardaron más, hasta 0.59s). Por lo tanto, el coste computacional está justificado: a cambio de 0.1 segundos extra, obtenemos una gran mejora en el ARI (+0.16 puntos).

Si hablamos de los hiperparámetros más concretamente: - *n_neighbors* en afinidad nn: Con vecinos bajos (5, 10, 15), el ARI es muy pobre (rondando el 0.1 - 0.2), ya que las proyecciones de UMAP tienen densidades variables, y si conectamos muy pocos vecinos, el grafo de un mismo dígito se divide en miniclusters. Al subir a nn=50, garantizamos que todo el grupo de un mismo número quede fuertemente entrelazado, disparando el ARI al 0.64.

- *gamma* en afinidad RBF: A mayor gamma, el radio de influencia es más pequeño y estricto. Vemos que gamma=1.0 (0.8104) supera a gamma=0.1 (0.7861), ya que como UMAP agrupa los datos en clústeres muy compactos y densos, un gamma más alto (radio pequeño) es mejor para no cruzar el vacío y conectar erróneamente clústeres de números diferentes que estén cerca.

1.3. DBSCAN

1.3.1. Escenarios sintéticos

Basándonos en la heurística del codo de los gráficos de k-distancias, he seleccionado configuraciones óptimas y valores intencionadamente estrictos o permisivos para evaluar la sensibilidad del hiperparámetro eps:

- Blobs: El valor óptimo es eps=0.3. Una configuración demasiado estricta (eps=0.1) fragmenta las agrupaciones y genera ruido artificial.
- Moons: El radio óptimo es eps=0.2. Si usamos el valor estricto (eps=0.1), el vecindario no alcanza la distancia necesaria y rompe la continuidad de las lunas.
- Circles: El ajuste óptimo es eps=0.2. Un parámetro demasiado permisivo (eps=0.4) amplía tanto el radio que tiende a saltar el vacío y fusionar los anillos concéntricos.
- Densidad variable: La gráfica no muestra un codo claro, confirmando que no hay un eps global perfecto. Un valor estricto (eps=0.2) destruye el clúster disperso marcándolo como ruido, mientras que un valor permisivo (eps=0.6) fusiona todos los clústeres en uno solo.
- Anisotrópico+ruido: A diferencia del resto, aquí el valor óptimo es el estricto (eps=0.1), ya que es el único que logra aislar el ruido de fondo. El valor permisivo (eps=0.3) falla porque absorbe erróneamente ese ruido dentro de los clústeres principales. Para complementarlo, es el único caso en el que he subido el mínimo de samples necesarios a 10.

Tabla 5: DBSCAN sobre escenarios sintéticos (mejor configuración por escenario)

Escenario	**eps**	**min_samples**	**N Clusters**	**% Ruido**	**ARI**
Blobs	0.30	5	3	0.00	1.0000
Blobs	0.10	5	3	1.93	1.0000
Moons	0.20	5	2	0.00	1.0000
Moons	0.10	5	2	1.33	1.0000
Circles	0.20	5	2	0.00	1.0000
Circles	0.40	5	2	0.00	1.0000
Densidad variable	0.20	5	8	6.55	0.9538
Densidad variable	0.60	5	1	0.27	0.0000
Anisotrópico+ruido	0.30	10	2	5.73	0.5323
Anisotrópico+ruido	0.15	3	8	8.73	0.9789

Comparando estos resultados con los de los métodos anteriores, podemos observar en cuales de los escenarios DBSCAN supera a K-Means y Spectral.

Al igual que lo que pasaba en la comparación de Spectral previa, DBSCAN supera ampliamente a K-Means en los escenarios no convexos (Moons y Circles), logrando un ARI perfecto de 1.0000. Al

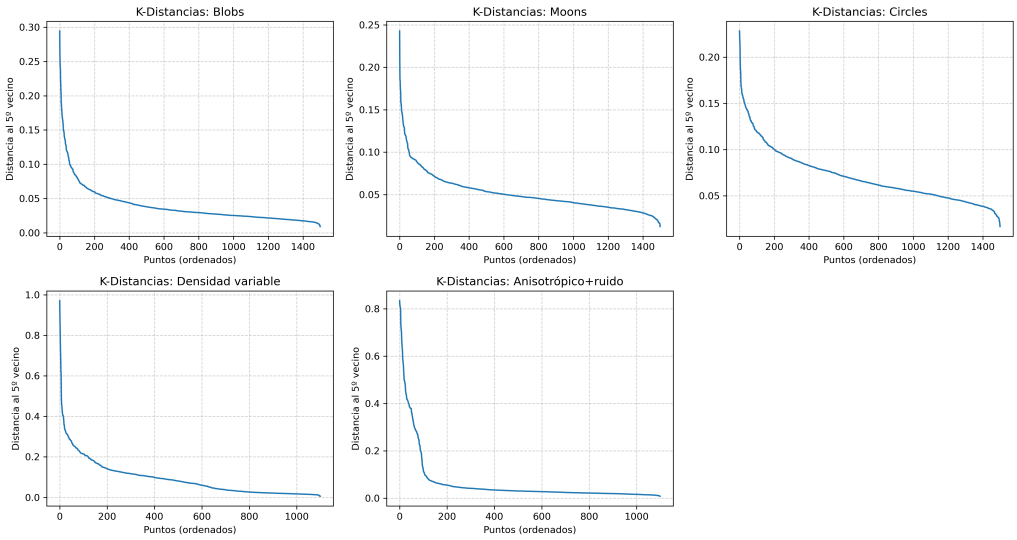


Figura 4: K-Distancias sobre los 5 escenarios sintéticos

basarse en la conectividad por densidad, DBSCAN puede moverse por el anillo o la luna sin importar su forma geométrica.

El apartado en el que sí que ofrece una novedad respecto a Spectral, es en el escenario Anisotrópico + ruido. Ni K-Means ni Spectral tienen el concepto de ruido, por lo que están obligados a asignar los puntos aleatorios de fondo a alguno de los clústeres principales, ensuciando los resultados. DBSCAN, en cambio, identifica activamente los puntos aislados, los aísla (etiqueta -1) y logra extraer los clústeres reales con un ARI casi perfecto (0.9789 con eps=0.1).

Sin embargo, si nos fijamos en el escenario de Densidad variable, observamos que DBSCAN es incapaz de segmentarlo correctamente en ambas configuraciones de prueba. Esto se debe al utilizar un umbral de densidad global, que provoca que sea imposible trazar una frontera de densidad única que funcione simultáneamente para regiones muy densas y muy dispersas:

- Si usamos un eps pequeño (0.2), el algoritmo captura bien los clústeres densos, pero los puntos del clúster disperso están tan separados entre sí que los fragmenta en múltiples subclústeres (encuentra 8 clústeres en total) y marca un 6.55

- Si usamos un eps grande (0.6) para intentar que el vecindario alcance a los puntos dispersos, el radio es tan amplio que termina puenteando el espacio entre los clústeres densos, fusionando todo en un único gran clúster (N° Clusters = 1, ARI = 0.0000).

Como último escenario, el anisotrópico genera resultados con mucho ruido (tanto con parámetros altos como con bajos). Estos puntos de ruido coinciden con los puntos de ruido reales, ya que con la mejor configuración (eps=0.1, min_samples=5), DBSCAN aísla exactamente un 8.73

1.3.2. Digits

Tabla 6: DBSCAN sobre Digits (selección representativa)

eps	min_samples	N Clusters	% Ruido	ARI	NMI
0.15	5	14	0.00	0.8419	0.8994
0.15	10	14	0.00	0.8419	0.8994
0.25	5	10	0.00	0.7119	0.8609
0.35	10	10	0.00	0.7119	0.8609
0.45	5	8	0.00	0.5508	0.8241
Mejor	0.15	14	0.00	**0.8419**	**0.8994**

Sobre Digits, vemos que el mejor ARI (0.8419) se consigue con el radio de vecindad más estricto (eps=0.1), independientemente de si pedimos 5 o 10 min_samples. Sin embargo, si nos fijamos en la

columna del número de clústeres, DBSCAN encuentra 14 clústeres, no 10. Esto se debe al tener un radio tan pequeño, donde DBSCAN es muy prudente y si UMAP ha dividido un mismo número en dos islas muy separadas, DBSCAN con eps=0.1 las mantendrá como clústeres separados. Al no fusionar islas erróneas, provoca que obtengamos un ARI tan alto.

Al aumentar el radio a eps=0.2 o 0.3, DBSCAN logra exactamente los 10 clústeres esperados. Sin embargo, el ARI baja a 0.7119. Esto indica que, al aumentar el vecindario para poder fusionar las sub-islas del mismo número, el radio se vuelve lo suficientemente grande como para empezar a fusionar ligeramente clústeres de números distintos que UMAP dejó demasiado cerca.

En casos más extremos, si relajamos el radio hasta eps=0.4, el algoritmo empieza a fusionar demasiadas cosas. El número de clústeres cae a 8 y el ARI se desploma a 0.5508. El vecindario es tan amplio que está puenteando el espacio entre números completamente diferentes y obteniendo unos resultados pésimos.

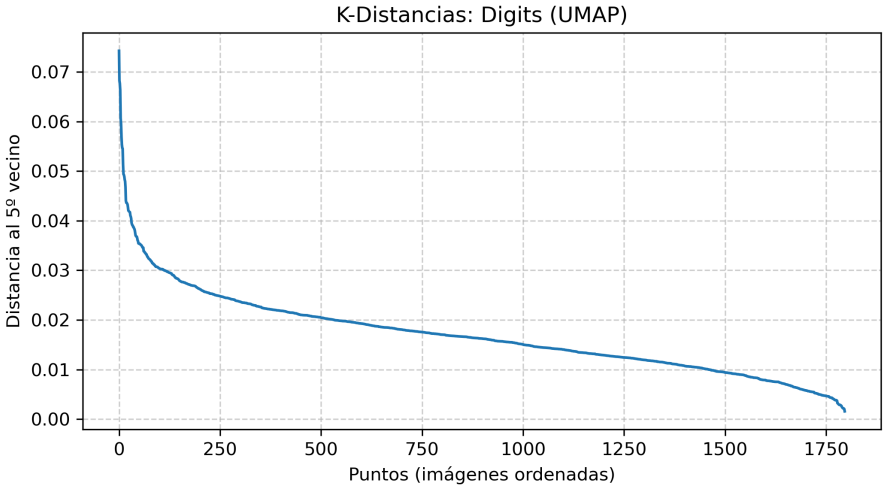


Figura 5: K-Distancias sobre Digits

1.4. OPTICS

1.4.1. Escenarios sintéticos

Se presta especial atención al escenario de **densidad variable**, donde DBSCAN falla por usar un único ε global.

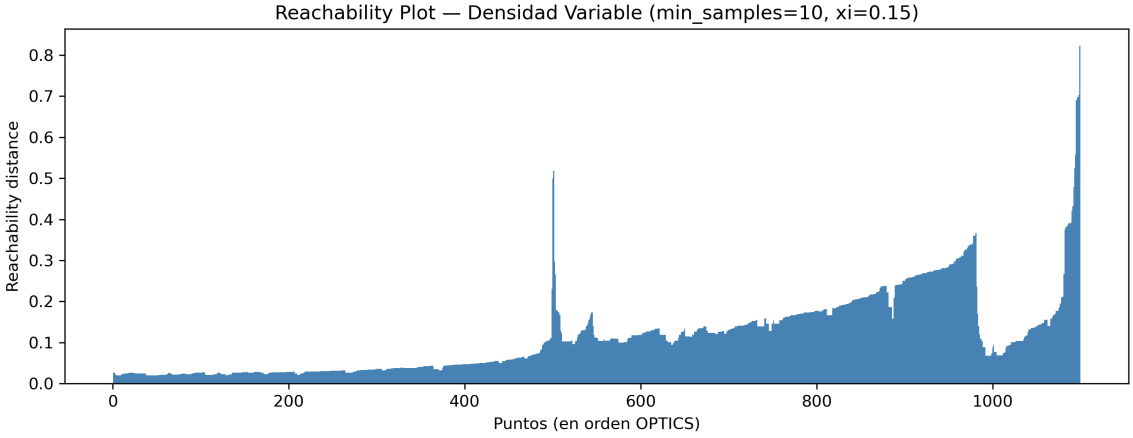


Figura 6: Reachability plot – escenario de densidad variable

Tabla 7: OPTICS sobre el escenario de densidad variable

min_samples	Método	xi / eps	N Clusters	% Ruido	ARI
5	xi	0.05	63	55.4	0.0385
10	xi	0.10	5	93.9	0.3575
10	xi	0.15	2	45.4	0.9973
5	dbscan	0.15	7	12.6	0.9579
10	dbscan	0.25	3	9.3	0.9959
10	dbscan	0.35	3	2.0	0.9959
Mejor	**OPTICS (xi)**	**0.15**	**2**	**45.4**	**0.9973**

Tabla 8: OPTICS – mejor resultado por escenario sintético

Escenario	Config	N Clusters	% Ruido	ARI
Blobs	xi=0.15, min_samp=10	3	0.0	1.0000
Moons	eps=0.10, min_samp=10	2	2.9	1.0000
Circles	eps=0.20, min_samp=10	2	0.1	1.0000
Densidad variable	xi=0.15, min_samp=10	2	45.4	0.9973
Anisotrópico+ruido	eps=0.20, min_samp=10	3	7.9	0.9746

El escenario de densidad variable es donde OPTICS demuestra su ventaja más clara sobre DBSCAN. Con DBSCAN el mejor ARI obtenido fue 0.9538 usando eps=0.2, pero con un radio tan pequeño solo detectaba 8 clusters en lugar de 3, fragmentando el cluster disperso. OPTICS con xi=0.15 y min_samples=10 alcanza ARI=0.9973 detectando exactamente 3 clusters, superando claramente a DBSCAN.

Mirando el reachability plot de abajo, los tres valles correspondientes a los tres clusters reales son perfectamente identificables. El primer valle, muy profundo y estrecho entre los puntos 0-500, corresponde al cluster denso centrado en [0,0]. El segundo valle, más elevado y amplio entre los puntos 500-1000, corresponde al cluster disperso centrado en [6,0]. El tercer valle, visible como la bajada brusca alrededor del punto 975, corresponde al cluster pequeño y denso centrado en [3,4].

Si comparamos los métodos de extracción, los resultados muestran que su rendimiento depende del escenario. Para densidad variable, xi=0.15 con min_samples=10 obtiene ARI=0.9973, ligeramente superior al mejor dbscan (ARI=0.9959 con eps=0.25 o eps=0.35). Sin embargo, xi es mucho más sensible a su parámetro: con xi=0.05 el algoritmo fragmenta en 63 clusters (ARI=0.038). dbscan dentro de OPTICS es más estable, ya que con eps=0.25 y eps=0.35 ambos dan ARI$\sim$0.996. Para el resto de escenarios, dbscan es el método superior (Moons y Circles alcanzan ARI=1.00 con dbscan).

En los escenarios más simples, Blobs con xi=0.15 alcanza ARI=1.00 sin ningún punto de ruido. El escenario anisotrópico con ruido obtiene ARI=0.9746 con un 37.9 % de ruido, un resultado razonable considerando que el dataset contiene 100 puntos de ruido artificial y clusters con forma elongada que dificultan la detección por densidad.

1.4.2. Digits

Tabla 9: OPTICS sobre Digits (selección representativa)

min_samples	xi	N Clusters	% Ruido	ARI	NMI
5	0.05	135	33.2	0.1291	0.6378
10	0.05	68	40.5	0.2288	0.6967
10	0.15	29	61.9	0.7629	0.8234
20	0.05	25	49.0	0.5899	0.8219
20	0.10	15	63.1	0.8623	0.9146
Mejor	**0.15**	**13**	**26.7**	**0.9455**	**0.9482**

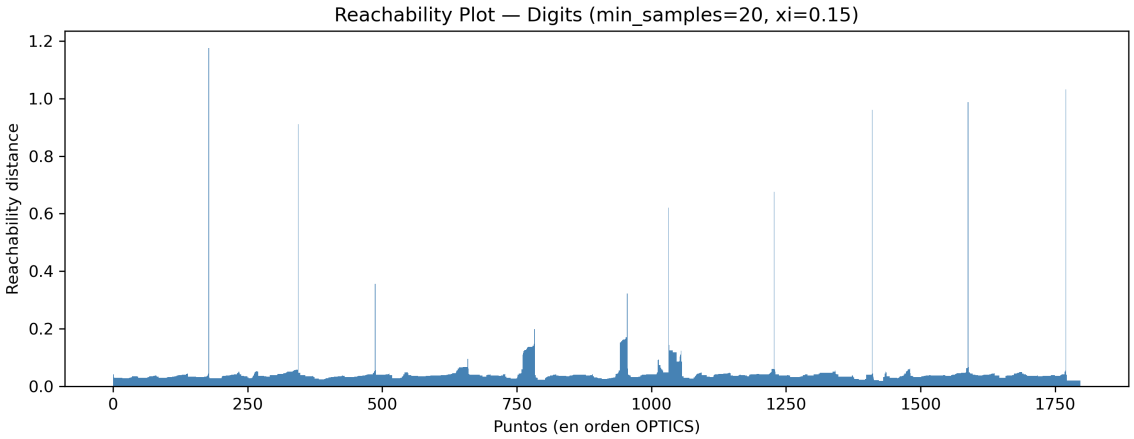


Figura 7: Reachability plot – Digits

Los resultados muestran una tendencia clara donde aumentar *min_samples* mejora consistentemente el ARI. Con *min_samples*=5, los resultados son pobres independientemente de *xi*, con el mejor ARI de apenas 0.1744. Esto se debe a que con tan pocos vecinos requeridos, OPTICS es demasiado permisivo y fragmenta en 94-135 clusters, muy lejos de los 10 reales. A medida que se sube *min_samples* a 10, 20 y 30 el algoritmo se vuelve más exigente para formar clusters y los resultados mejoran progresivamente.

El mejor resultado es *min_samples*=20, *xi*=0.15 con ARI=0.9455 y NMI=0.9482, detectando 13 clusters con un 26.7 % de ruido. Es un buen resultado, aunque ligeramente por debajo del mejor DBSCAN (ARI=0.8419). Con *min_samples*=30 se obtiene ARI=0.9195 con exactamente 11 clusters y solo 8.8 % de ruido, lo que indica que el algoritmo está encontrando una estructura muy cercana a la real aunque sacrifica algo de ARI al fusionar ligeramente algunos dígitos.

Respecto a *xi*, un valor más alto exige pendientes más pronunciadas para separar clusters, evitando fragmentaciones espurias, por ejemplo, con *min_samples*=20, subir *xi* de 0.05 a 0.15 mejora el ARI de 0.5899 a 0.9455. Sin embargo con *min_samples*=30, *xi*=0.10 y *xi*=0.15 dan exactamente el mismo resultado (ARI=0.9195, 11 clusters), lo que sugiere que a ese nivel de *min_samples* el *reachability plot* ya tiene valles tan bien definidos que el umbral exacto de *xi* deja de importar.

1.5. Comparativa Final – Parte 1

1.5.1. Escenarios sintéticos

Tabla 10: Resumen ARI – todos los métodos en todos los escenarios sintéticos

Escenario	**K-Means**	**Spectral**	**DBSCAN**	**OPTICS**
Blobs	1.0000	1.0000	1.0000	1.0000
Moons	0.4785	1.0000	1.0000	1.0000
Circles	-0.0006	1.0000	1.0000	1.0000
Densidad variable	0.8150	0.9567	0.9538	0.9973
Anisotrópico+ruido	0.8507	0.8405	0.9789	0.9746

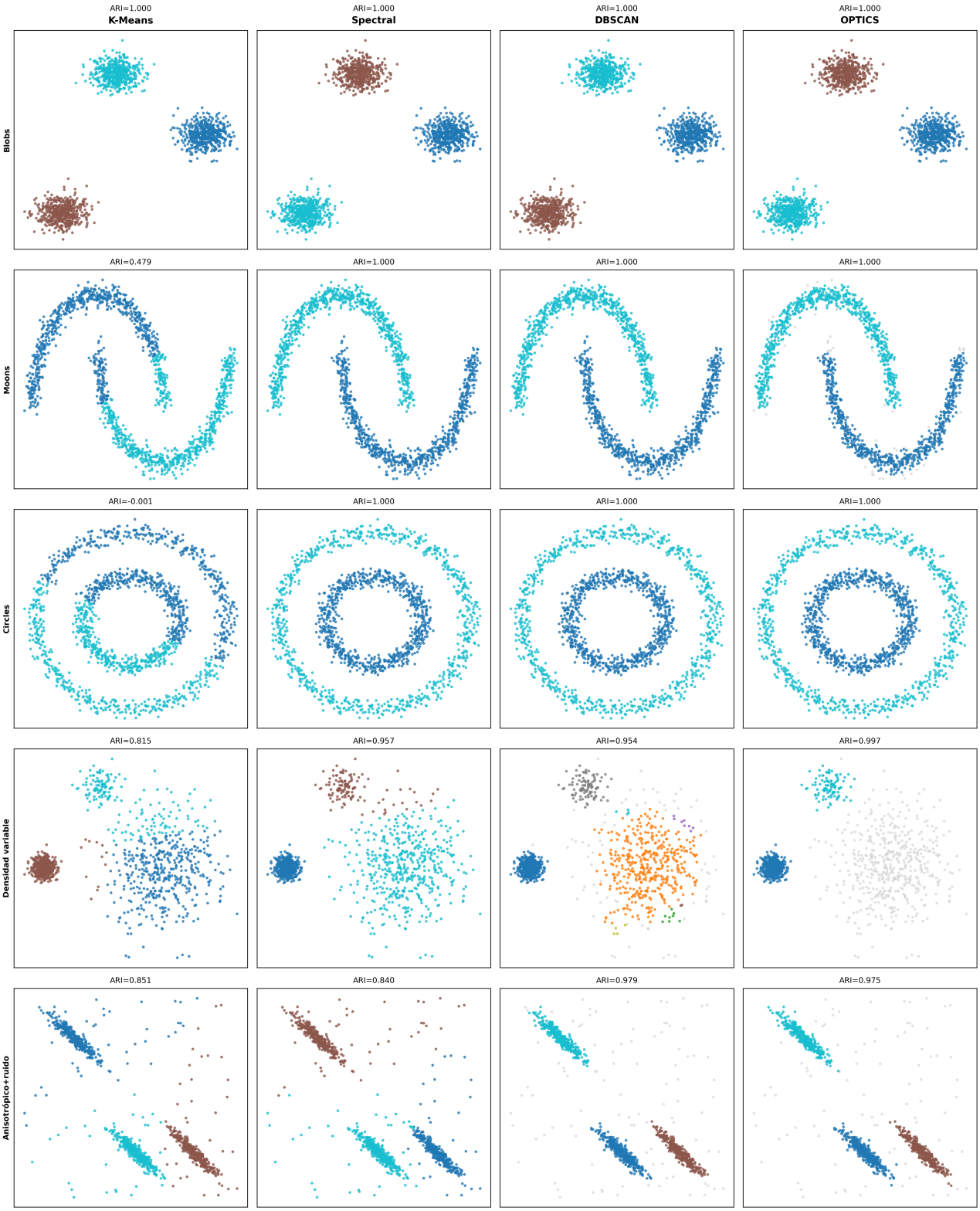


Figura 8: Visualización de todos los métodos en todos los escenarios sintéticos

1.5.2. Digits

OPTICS es el claro ganador en Digits con ARI=0.9455, superando a DBSCAN (0.8419), K-Means (0.8248) y Spectral (0.8104). La diferencia es especialmente notable respecto a Spectral y K-Means, que se quedan en torno a 0.81-0.82. OPTICS detecta 13 clusters con un 26.7 % de ruido, lo que indica

Tabla 11: Comparativa final sobre Digits

Método	Config	N Clusters	% Ruido	ARI	NMI	Tiempo (s)
K-Means	$K = 10$	10	0.0 %	0.8248	0.8912	0.0912
Spectral	rbf, $\gamma = 1{,}0$	10	0.0 %	0.8104	0.8798	0.3957
DBSCAN	eps=0.1, ms=5	14	0.0 %	0.8419	0.8994	0.0134
OPTICS	ms=20, xi=0.15	13	26.7 %	0.9455	0.9482	1.5087

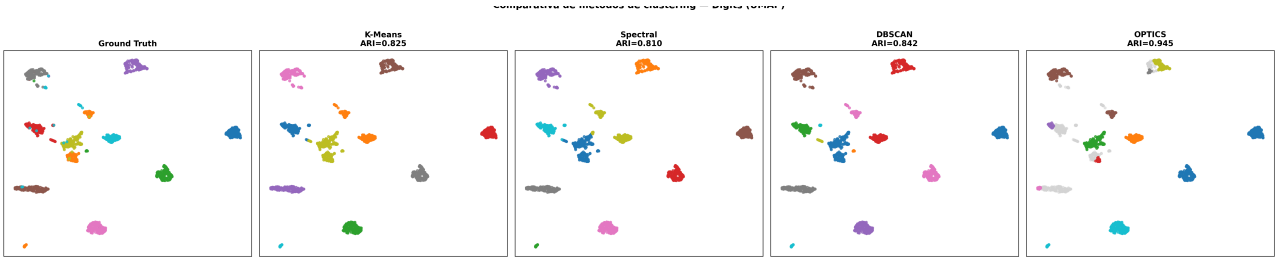


Figura 9: Comparativa de métodos sobre Digits (UMAP 2D)

que descarta puntos ambiguos entre dígitos en lugar de asignarlos forzosamente, y eso mejora la calidad de los clusters que sí forma. Esto se puede ver claramente en la gráfica comparativa 5x5 de abajo.

Por otro lado, fijándonos en los escenarios sintéticos previamente, estos mostraban que OPTICS es el método más versátil, siendo es el único que obtiene ARI cercano a 1.0 en todos los escenarios incluyendo densidad variable, donde K-Means y DBSCAN fallaban. Sin embargo, los sintéticos también mostraban que OPTICS es muy sensible a sus parámetros, con resultados que van de ARI=0.04 a ARI=0.997 según la configuración. En ese sentido, el buen resultado en Digits no estaba garantizado y dependía de encontrar los parámetros adecuados. Lo que sí podía anticiparse es que K-Means tendría dificultades, ya que Digits proyectado con UMAP genera formas no convexas similares a Moons o Circles, donde K-Means obtuvo ARI=0.4785 y -0.0006 respectivamente.

Si comparamos las métricas (internas y externas), observamos que coinciden de manera parcial. K-Means obtiene el mejor Silhouette (0.7737) pero no el mejor ARI (0.8248), lo cual se debe al optimizar directamente la separación esférica de clusters, produciendo asignaciones limpias que el Silhouette premia, pero no necesariamente correctas respecto a las etiquetas reales. OPTICS obtiene el mejor ARI (0.9455) pero no se puede calcular su Silhouette de forma directa porque tiene puntos de ruido sin asignar, lo que distorsiona la métrica.

En el caso de no haber etiquetas, la elección dependería del contexto. Si los datos tienen estructura geométrica compleja o densidad variable, como ocurre con Digits proyectado en UMAP, OPTICS con una exploración de *min_samples* y *xi* es la opción más potente aunque costosa de calibrar. Si se busca rapidez y simplicidad, DBSCAN es una buena alternativa, ya que es el más rápido (0.013s frente a 1.5s de OPTICS), fácil de interpretar con el *k-distances plot*, y en Digits obtiene un ARI competitivo. K-Means quedaría como opción solo si se sabe de antemano que los clusters son aproximadamente convexos y el número de clusters es conocido. Spectral es potente pero su coste computacional (0.4s) y sensibilidad a los parámetros lo hacen menos práctico sin etiquetas para validar. En resumen, para un dataset desconocido sin etiquetas la recomendación sería empezar con DBSCAN por su interpretabilidad y bajo coste, y escalar a OPTICS si los resultados visuales sugieren densidad variable o estructuras complejas.

2. Parte 2: Detección de Anomalías

2.1. Dataset y Exploración

Tabla 12: Características del dataset de anomalías

Característica	Valor
Muestras (total)	100,655
Features	41
Anomalías (n)	3,377
Anomalías (%)	3.36 %
Varianza PCA 2D	27.30 %
Fuente	KDD Cup 99

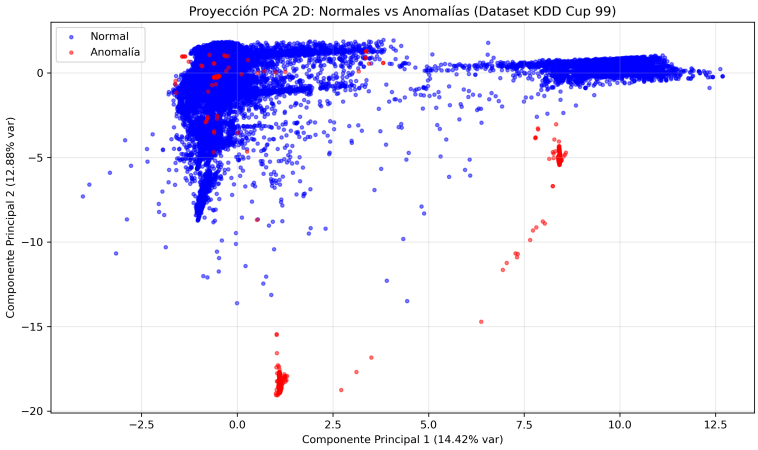


Figura 10: Proyección PCA 2D del dataset de anomalías

2.2. Gaussian Mixture Models (GMM)

2.2.1. Selección de componentes y covarianza

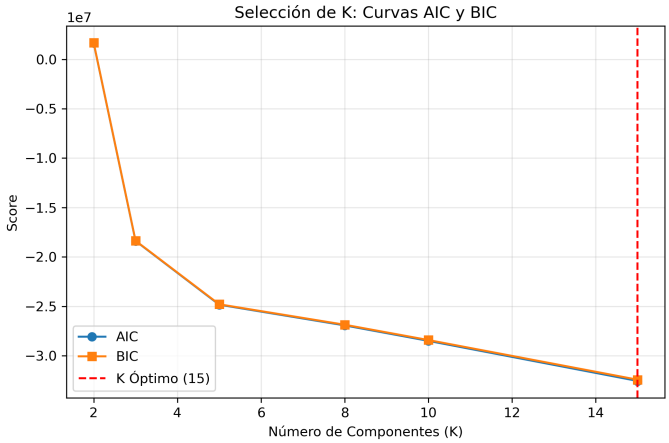


Figura 11: Selección de K con BIC/AIC

K óptimo (BIC): 15

2.2.2. GMM como detector de anomalías

Tabla 13: GMM como detector de anomalías

cov_type	Percentil	Prec	Recall	F1	ROC-AUC	AP
full	95	0.0250	0.0373	0.0300	0.2321	0.0347
full	97	0.0368	0.0329	0.0347	0.2321	0.0347
full	99	0.0914	0.0272	0.0420	0.2321	0.0347
tied	95	0.0546	0.0814	0.0654	0.5112	0.0486
tied	97	0.0825	0.0737	0.0778	0.5112	0.0486
tied	99	0.1271	0.0379	0.0584	0.5112	0.0486
diag	95	0.1341	0.1999	0.1605	0.2783	0.0549
diag	97	0.0536	0.0480	0.0506	0.2783	0.0549
diag	99	0.1132	0.0338	0.0520	0.2783	0.0549
spherical	95	0.0805	0.1199	0.0963	0.3199	0.0523
spherical	97	0.1328	0.1187	0.1254	0.3199	0.0523
spherical	99	0.0844	0.0252	0.0388	0.3199	0.0523
Mejor	**95 (diag)**	**0.1341**	**0.1999**	**0.1605**	**0.2783**	**0.0549**

Para este análisis, exploramos con la curva BIC $K \in \{2, 3, 5, 8, 10, 15\}$ donde selecciona $K = 15$ como óptimo. Sin embargo, $K = 15$ es el valor máximo del rango explorado propuesto por el enunciado, lo que indica que el BIC seguía descendiendo sin haber encontrado un mínimo real. Esto sugiere que la estructura del subconjunto SA requiere más componentes para ser modelada adecuadamente, y que el K óptimo real podría estar por encima de 15.

Una vez asignamos el número de componentes óptimo, iteramos sobre tipos de covarianza para comprobar su rendimiento. Aunque ninguna configuración destaca especialmente, *tied* obtiene el mejor ROC-AUC (0.51) y el mejor F1 con percentil 95 (0.065), mientras que *diag* consigue el mejor Recall individual (0.1999 con percentil 95). *full* es la peor en todas las métricas, con ROC-AUC de apenas 0.23. La diferencia entre tipos es moderada pero consistente, siendo *tied* la opción más estable. Sin embargo, Recall será la métrica que usaremos como principal en el análisis general posterior del bloque.

Otro de los aspectos a comentar es el uso de GMM como detector de anomalías. El dataset SA tiene un 3.36 % de anomalías reales, mientras que los percentiles evaluados (95, 97, 99) marcan como anomalía el 5 %, 3 % y 1 % de los datos respectivamente, siendo el percentil 97 el más cercano al ratio real, pero no se traduce en mejores métricas: en *diag* el mejor Recall se obtiene con percentil 95, y en *tied* con percentil 97. Esto indica que el GMM no está bien calibrado para este problema independientemente del umbral elegido.

Los valores de ROC-AUC oscilan entre 0.23 y 0.51, todos cercanos o por debajo del azar (0.5), con lo cual sabemos que la log-likelihood del GMM no discrimina bien entre tráfico normal y ataques. La causa probable es que el modelo se entrena sobre datos mezclados (normal y ataques), por lo que aprende también la distribución de los ataques y les asigna una densidad que no es despreciable. En consecuencia, el score de anomalía (`-score_samples`) no separa limpiamente ambas clases. Además, el Recall máximo alcanzado es 0.1999, lo que significa que el modelo solo detecta aproximadamente 1 de cada 5 ataques reales. En este contexto de detección de intrusiones esto es un rendimiento muy bajo, y confirma que GMM no es adecuado como detector de anomalías en este dataset sin un preprocesamiento o separación previa entre datos normales y anomalías.

2.3. Isolation Forest

El parámetro *contamination* es el que más impacto tiene en los resultados. Observamos una relación directa donde a mayor *contamination*, mayor Recall y menor Precision. Con cont=0.01 se obtiene el mejor F1 (0.4197) y la mejor Precision (0.91), pero un Recall bajo (0.27), detectando solo 1 de cada 4 ataques. Con cont=0.20 el Recall sube a 0.9959 pero la Precision cae a 0.1671. Esta dinámica se explica directamente por el desbalanceo del dataset, ya que la tasa real de anomalías es del 3.36 %, por lo que cont=0.01 subestima ese ratio y cont=0.20 lo sobreestima. Ningún valor del rango explorado coincide exactamente con la tasa real, aunque cont=0.05 sería el más cercano. Sin embargo, cont=0.05

Tabla 14: Efecto de `contamination` en Isolation Forest

contamination	n_estimators	Prec	Recall	F1	ROC-AUC	AP
0.01	100	0.9116	0.2718	0.4188	0.9411	0.4524
0.05	100	0.1848	0.2745	0.2209	0.9411	0.4524
0.10	100	0.1684	0.4957	0.2514	0.9411	0.4524
0.15	100	0.2225	0.9947	0.3636	0.9411	0.4524
0.20	100	0.1671	0.9959	0.2861	0.9411	0.4524
Efecto de `n_estimators` (con mejor contamination):						
0.01	50	0.9096	0.2712	0.4179	0.9273	0.4246
0.01	150	0.9086	0.2710	0.4174	0.9363	0.4412
0.01	200	0.9136	0.2724	0.4197	0.9426	0.4576
0.01	300	0.9086	0.2710	0.4174	0.9447	0.4628
Mejor	**200**	**0.9136**	**0.2724**	**0.4197**	**0.9426**	**0.4576**

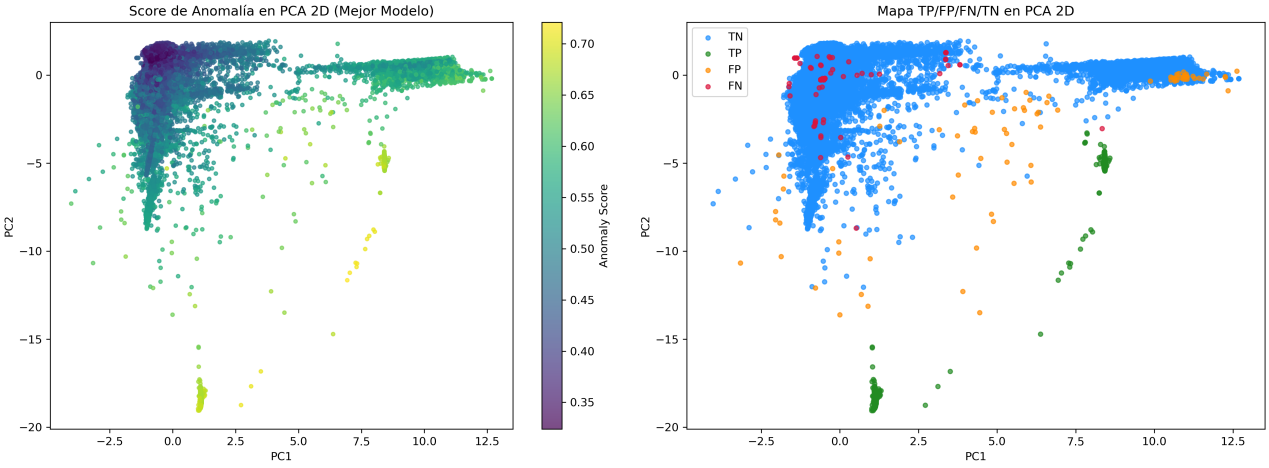


Figura 12: Isolation Forest – score de anomalía y clasificación

no es el óptimo ni por F1 ni por Recall, por lo que sabemos que en datasets desbalanceados la métrica de selección condiciona fuertemente la configuración elegida. Dado que en detección de intrusiones el Recall es la métrica más importante, la configuración más relevante para este problema es cont=0.20, n_estimators=100, a pesar de no ser la óptima por F1.

Para poder realizar una exploración del número de estimadores algo más significativa, he escogido contamination = 0.01 como el mejor valor basándome en F1-Score y no en Recall (pese a que Recall será el utilizado para esocger la mejor configuración), ya que los valores de Recall con un nivel de contamination más alto (como 0.2) ya son suficientemente elevados como para que una exploración de los estimadores no varíe mucho sus resultados y no se pueda apreciar el cambio de aumentar o disminuir la cantidad.

Para el caso comentado del número de estimadores, la exploración con cont=0.01 (donde variamos n_estimators entre 50 y 300) produce cambios pequeños: el F1 oscila entre 0.4174 y 0.4197, y el ROC-AUC entre 0.9273 y 0.9447. Esto indica que el modelo converge con relativamente pocos árboles y que añadir más estimadores no aporta mejora significativa en este dataset. Con cont=0.20 tenemos estabilidad, ya que de haber variado sus estimadores, todas las supuestas configuraciones de n_estimators producirían exactamente los mismos resultados.

El ROC-AUC se mantiene estable en torno a 0.94 independientemente de contamination, lo que indica que el ranking continuo del score es robusto y el modelo distingue bien entre normal y ataque en términos de separación. Sin embargo, el Avg Precision (0.45-0.46) es moderado para un ROC-AUC tan alto, lo que sugiere que en los umbrales de alta precisión el modelo tiene más dificultades, es decir, los ataques más difíciles no están completamente aislados del tráfico normal. En el espacio

PCA, las anomalías detectadas se concentran en regiones periféricas y de baja densidad (tipo de estructura que Isolation Forest explota mediante particiones aleatorias cortas), más concretamente en la zona baja del gráfico con colores más amarillentos. Los FP del segundo gráfico, aparecen dispersos en zonas fronterizas entre ambas clases, lo que es coherente con la baja precisión observada cuando contamination es alta.

2.4. DBSCAN como Detector de Anomalías

Tabla 15: DBSCAN como detector de anomalías

eps	min_samples	% Ruido	Prec	Recall	F1
0.5	5	9.51	0.0127	0.0361	0.0188
1.0	5	3.05	0.0294	0.0267	0.0279
1.5	5	1.54	0.0393	0.0181	0.0248
0.5	10	12.85	0.0100	0.0382	0.0158
1.0	10	4.22	0.0254	0.0320	0.0283
1.5	10	2.19	0.0377	0.0246	0.0297
Mejor	**1.5**	**2.19**	**0.0377**	**0.0246**	**0.0297**

El k-distance plot (k=5) muestra una curva casi completamente plana desde el punto 0 hasta aproximadamente el punto 100.000, con un único salto drástico en los últimos puntos. Esto indica que la gran mayoría del dataset forma una región extremadamente densa y compacta, sin una separación clara entre tráfico normal y ataques. El codo ideal para elegir eps debería ser un punto de inflexión suave que aquí prácticamente no existe, lo que nos anticipa que cualquier valor de eps razonable va a resultar problemático.

Todas las configuraciones evaluadas presentan métricas muy bajas, con F1 máximo de 0.0297 y Recall máximo de 0.0382. El problema principal no es que DBSCAN etiquete demasiados puntos normales como ruido (los porcentajes de ruido son bajos, entre 1.54 % y 12.85 %), sino que no detecta las anomalías, ya que el Recall es extremadamente bajo en todas las configuraciones, lo que significa que la inmensa mayoría de los 3.377 ataques reales quedan absorbidos dentro del cluster principal junto al tráfico normal. Esto se debe a que la tasa de anomalías es del 3.36 % sobre 100.655 muestras, por lo que hay 3.377 ataques. Aunque sean una minoría, en el espacio de dimensiones pueden estar suficientemente cerca de puntos normales como para cumplir los criterios de densidad de DBSCAN y ser incorporados al cluster mayoritario, ya que DBSCAN agrupa por proximidad.

Pese a ser un método que falle en este problema, en el caso de que se realizase un preprocesado diferente podría mejorar. Con 41 features, las distancias euclidianas pierden significado y todo parece igualmente denso, como refleja la curva k-distance casi plana. Aplicar una reducción de dimensionalidad (PCA o UMAP) antes de DBSCAN concentraría la estructura real del dato en menos dimensiones, haría los clusters más separables y el codo del k-distance plot sería más pronunciado y útil para elegir eps. En su estado actual, DBSCAN no es adecuado como detector de anomalías para este dataset.

2.5. Evaluación Final – Parte 2

Tabla 16: Comparativa final – detección de anomalías

Modelo	Config	Prec	Recall	F1	ROC-AUC	AP
GMM	K=15, diag, pct=95	0.1341	0.1999	0.1605	0.2783	0.0549
Isolation Forest	n_est=100, cont=0.20	0.1671	0.9959	0.2861	0.9411	0.4524
DBSCAN	eps=0.5, min_samp=10	0.0100	0.0382	0.0158	N/A	N/A

En el caso de nuestro dataset (detección de intrusiones de red), el Recall es la métrica más importante. Un ataque no detectado (falso negativo) tiene consecuencias potencialmente graves, mientras que una falsa alarma (falso positivo) es simplemente un coste operativo asumible. Bajo este criterio,

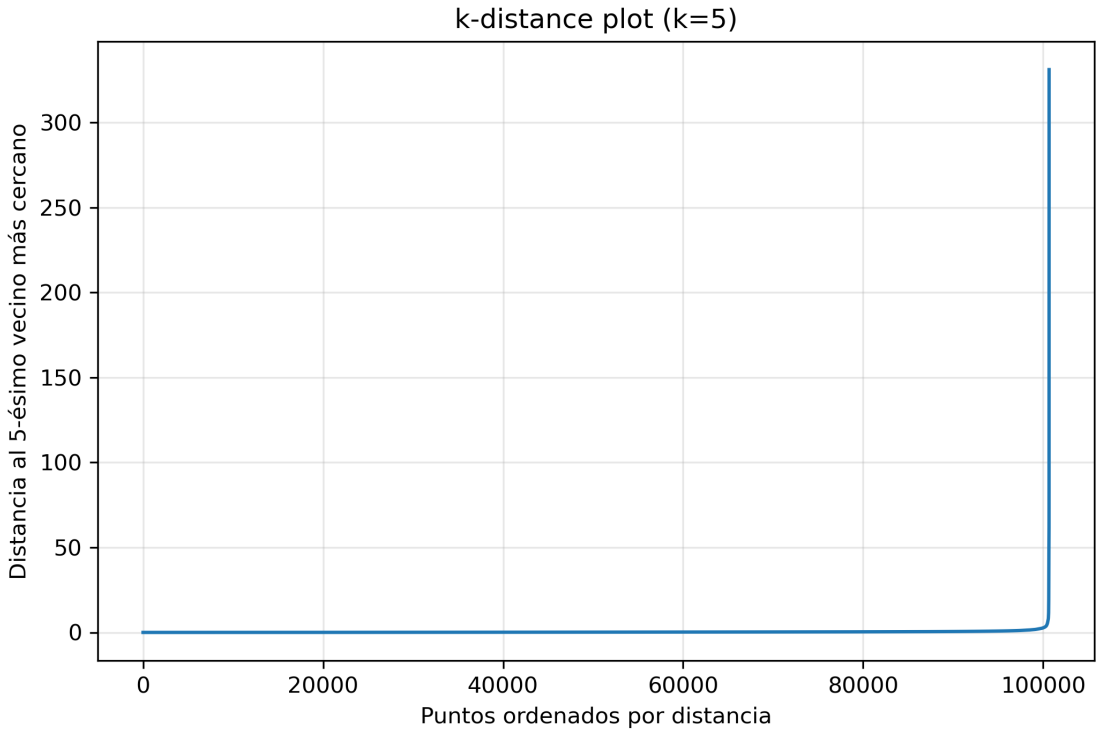


Figura 13: k-distance plot (k=k)

Isolation Forest es el claro ganador con un Recall de 0.9959, frente al 0.1999 de GMM y el 0.0382 de DBSCAN.

Isolation Forest detecta 3.363 de los 3.377 ataques reales, dejando escapar únicamente 14. El coste es alto en falsos positivos: 16.768 conexiones normales son marcadas como ataque, lo que explica la baja Precision (0.1671). Sin embargo, esto es una consecuencia directa del valor cont=0.20, que sobreestima el ratio real de anomalías (3.36 %). Su ROC-AUC de 0.9411 confirma que el score continuo del modelo es sólido y que con un threshold mejor calibrado los resultados mejorarían. Es el único modelo competitivo para este problema.

GMM detecta solo 675 ataques de 3.377, dejando escapar 2.702. Aunque su matriz de confusión muestra pocos falsos positivos (4.358) en relación a Isolation Forest, esto no es una virtud sino una consecuencia de que el modelo simplemente no identifica la mayoría de los ataques. Su ROC-AUC de 0.2783 confirma que el score de anomalía no discrimina bien entre clases, siendo el peor modelo en términos de ranking.

DBSCAN es el modelo con peor rendimiento global. Solo detecta 129 ataques de 3.377, con 12.807 falsos positivos, lo que resulta en la peor combinación posible: bajo Recall y baja Precision simultáneamente. Los valores N/A en ROC-AUC y Avg Precision reflejan además que DBSCAN no genera un score continuo, limitando su utilidad para ajustar el threshold según las necesidades del problema.

Isolation Forest es el único método adecuado para este dataset, tanto por diseño (está construido específicamente para detección de anomalías) como por resultados. GMM y DBSCAN no son competitivos en este contexto: GMM falla porque aprende también la distribución de los ataques al entrenarse con datos mezclados, y DBSCAN falla porque los ataques no forman regiones aisladas de baja densidad en un espacio de 41 dimensiones. En un escenario real de detección de intrusiones, solo Isolation Forest ofrecería una protección aceptable.

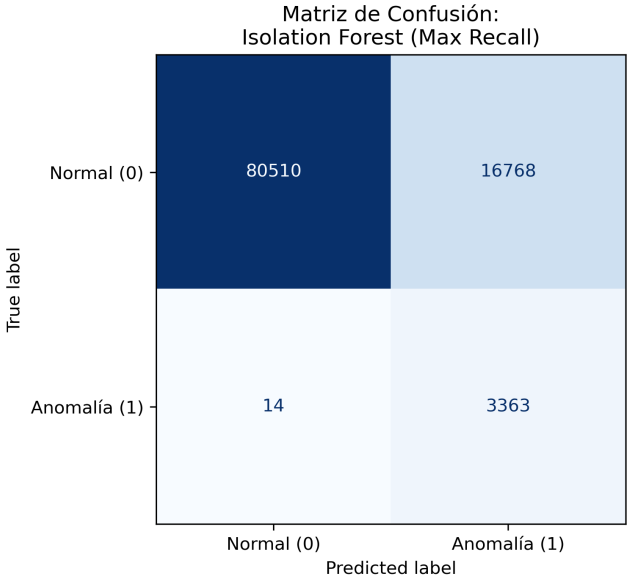


Figura 14: Matriz de confusión – mejor detector de anomalías

3. Conclusiones

- **Clustering – sintéticos:** En los escenarios sintéticos, la evaluación demuestra que la eficacia de cada método depende de la estructura de los datos. K-Means ha demostrado ser óptimo únicamente en estructuras convexas y esféricas como Blobs, fracasando en geometrías no convexas como Moons y Circles debido a su dependencia de la distancia euclídea y las fronteras lineales. Para estas formas complejas, tanto el Clustering Espectral como DBSCAN superaron ampliamente a K-Means, logrando segmentarlas perfectamente al aprovechar la continuidad espacial y la densidad. Sin embargo, DBSCAN fracasa ante el escenario de Densidad variable al requerir un umbral de densidad global, una limitación que supera OPTICS. En definitiva, OPTICS se ha consolidado como el método más versátil al obtener resultados excelentes en todas las estructuras evaluadas, aunque ha demostrado ser un algoritmo altamente sensible a la configuración de sus hiperparámetros.

- **Clustering – Digits:** En cuanto al dataset de Digits proyectado con UMAP, OPTICS también resulta ser el mejor método, alcanzando su mejor rendimiento con la configuración de min_samples=20 y xi=0.15. Con estos parámetros, el algoritmo logró un ARI sobresaliente de 0.9455, superando claramente a alternativas como DBSCAN, K-Means y Spectral Clustering. El éxito de OPTICS en este contexto se debe a su capacidad para identificar y descartar los puntos ambiguos situados entre los dígitos, tratándolos como ruido (26.7 %) en lugar de forzar su asignación, lo que mejora drásticamente la calidad y pureza de los 13 clústeres detectados. Por otro lado, si el contexto del problema exigiera priorizar la rapidez y simplicidad computacional, DBSCAN se presenta como una alternativa muy sólida y rápida frente al alto coste de tiempo y calibración que requiere OPTICS.

- **Detección de anomalías:** En el ámbito de la detección de anomalías, Isolation Forest se posiciona como el único método verdaderamente adecuado y competitivo para este problema. En un contexto de detección de intrusiones de red, donde maximizar el Recall es la máxima prioridad para evitar los graves riesgos de omitir un ataque (falsos negativos), este algoritmo logra un sobresaliente Recall de 0.9959, detectando 3.363 de los 3.377 ataques reales. Para ello, la configuración óptima es contamination=0.20 y n_estimators=100, la cual, aunque sobreestima la tasa real de anomalías (3.36 %) y sacrifica la precisión al generar numerosos falsos positivos , mantiene un elevado ROC-AUC (0.9411) que demuestra la solidez de su score continuo. Por el contrario, las alternativas evaluadas no han sido viables: GMM fracasa al entrenarse con datos mezclados y aprender erróneamente la distribución de los ataques, y DBSCAN fue el

peor modelo global al obtener casi todas las anomalías dentro del clúster mayoritario de tráfico normal, incapaz de separar por densidad en un espacio tan complejo de 41 dimensiones.

- **Métricas internas vs externas:** La comparativa entre métricas internas y externas nos idican que ambas coinciden solo de manera parcial. Se ha demostrado que una puntuación Silhouette alta no garantiza una segmentación correcta, como por ejemplo en K-Means, donde obtuvimos el mejor Silhouette al optimizar la separación esférica y compacta, pero no el mejor ARI, ya que esta métrica interna premia la estructura que el algoritmo busca por diseño, aunque no corresponda a la realidad de los datos. Esto sucede porque Silhouette se basa en distancias euclídeas, lo que puede dar resultados engañosos en escenarios con formas no convexas o densidades variables donde la métrica premia cortes lineales erróneos.
- **Aprendizajes:** El desarrollo de esta práctica nos permite comprender de manera estructural cada familia algorítmica. Por un lado, los métodos basados en centroides como K-Means resultan útiles para distribuciones convexas o esféricas, pero su dependencia geométrica de la distancia euclídea los vuelve ineficaces ante formas complejas. Para solucionar esto, los métodos como el Clustering Espectral capturan su continuidad espacial, aunque exigen un mayor coste computacional y presentan una fuerte sensibilidad a la configuración del grafo de similitud. Por otra parte, la familia basada en densidad es versátil, siendo DBSCAN una opción rápida e interpretable, pero fracasa ante densidades variables o espacios de alta dimensionalidad, siendo un obstáculo que OPTICS logra superar con gran éxito, a costa de requerir una exhaustiva y costosa exploración de sus parámetros. Finalmente, en la detección de anomalías, he aprendido que modelos generativos como GMM no son viables si se entrenan con datos mezclados al terminar aprendiendo la distribución del propio ataque, mientras que los algoritmos creados específicamente para este fin, como Isolation Forest, son imprescindibles por su capacidad para aislar las anomalías mediante particiones y maximizar la detección (Recall) en entornos críticos.